

Creating MongoDB Schemas and Models

The **DarshanEase – Temple Darshan Ticket Booking System** uses **MongoDB** with **Mongoose** to store and manage application data. Before implementing the backend functionality, schemas are created based on the **Entity Relationship (ER) Diagram** to define the structure of each collection in the database.

Each schema specifies the fields, data types, validation rules, and relationships required for storing information related to users, temples, darshan slots, bookings, and payments. Mongoose models are then created from these schemas to perform database operations such as creating, retrieving, updating, and deleting records.

The primary schemas implemented in the DarshanEase project include:

- **User Schema** – Stores user information such as name, email, phone number, password, and address.
- **Temple Schema** – Stores temple details including temple name, location, darshan timings, facilities, and images.
- **Darshan Slot Schema** – Stores available darshan dates, slot timings, ticket prices, and seat availability.
- **Booking Schema** – Stores booking information such as booking ID, user details, selected temple, selected slot, booking date, and payment status.
- **Payment Schema** – Stores payment transaction details including payment ID, amount, payment method, transaction ID, and payment status.

These schemas are implemented as **Mongoose Models**, which serve as the interface between the backend application and the MongoDB database. The models ensure consistent data storage, validation, and efficient execution of CRUD (Create, Read, Update, Delete) operations.

The following figure shows an example of a **Mongoose Schema** used in the **DarshanEase** backend application.

```
JS User.js X JS Admin.js JS Organizer.js JS Darshan.js JS
code > backend > models > JS User.js > ...
1  const mongoose = require('mongoose');
2
3  const UserSchema = new mongoose.Schema({
4    name: String,
5    email: { type: String, unique: true },
6    password: String,
7    userId: {
8      type: mongoose.Schema.Types.ObjectId,
9      ref: "User"
10   }
11 });
12
13 module.exports = mongoose.model('User', UserSchema);
14
```